Linked List-1

(scattered memory)

- Linked List → not continous memory allocation like array
  - ↳ large size can be created

↓

Linear data structure, where each element is a ~~seper~~ seperate object called a Node.

Agenda:

1. Intro: Use of Linked list over array,
2. Access kth index node
3. Search for element x.
4. Insert element x at position k.
5. Delete 1st occurence of x.

① Node Structure:

```
class Node {
    int data;        // To store the value
    Node next;       // to store the address of the next node.

    // Constructor to initialize a new node.
    Node (int x) {
        this.data = x;
        this.next = null;   // By default, a new node points to nothing.
    }
}
```

---

② Accessing the kth Node

T.C = O(k)

```
public static int getKthNode (Node head, int k){
    Node temp = head;          //start at the head
    for (int i = 0; i<k; i++){
        temp = temp.next;      // Move to the next node
    }
    return temp.data;          // Return the data found at that node.
}
```

Shallow copy

---

③ Searching for an Element x

T.C = O(n

```
public static boolean search(Node node, int x) {
    Node temp = head;
    while (temp != null) {          // Loop until the end of list
        if (temp.data == x) {
            return true;            // Found it
        }
        temp = temp.next;           // Move to next node
    }
    return false;                   // not found
}
```

(4) Insertion in Linked List

- inserting element x at position k.
- To insert a node, you don't shift elements like in an array. You simply change where the pointers point.
- **Case-1: Insert at Head (k=0)**: Make the new node point to the current head, then make the new node the new head.
- **Case2: Insert at Middle / End**: Stop at the node before the insertion point (k-1). Point the new node to the next node, and point the (k-1) node to the new node.

Code →

```
public static Node insert (Node head, int x, int k){
        Node newNode = new Node(x);

# Case-1: Insert at very beginning. (head changes)
        if (k==0) {
                newNode.next = head;
                return newNode; # This is the new head
        }
        Node temp = head;
# Move to the (k-1)th node.                    # Case: 2
        for (int i=0; i<k-1; i++){
                temp = temp.next;
        }

# Link the new node into the list
                newNode.next = temp.next;
                temp.next = newNode;
        return head;
}
```

5 Deleting the First Occurence of X

T.C = O(N)

→ To delete a node, you "Skip" it by making the node before it point directly to the node after it.

```
Public static Node delete (Node head, int x){
    if (head == null) return null;
```

Case: If head itself needs to be deleted.

```
    if ( head. data == x){ return head.next; }
                    // The 2nd node becomes the new head.

    Node temp = head;
```

Look for the node BEFORE the one we want to delete

```
    while ( temp.next != null){
        if (temp.next.data == x){
            temp.next = temp.next.next;  //Skip the node
            break; #Stop after deleting the 1st occurence.
        }
        temp = temp.next;
    }
    return head;
}
```